RISC-V Based MYTH Workshop

Microprocessor for You in Thirty Hours

Day 3: Digital Logic with TL-Verilog in Makerchip IDE

Workshop Certification Submission Report

Instructor:	Steve Hoover, Founder – Redwood EDA
Platform:	Makerchip IDE (makerchip.com)
HDL Used:	TL-Verilog (Transaction-Level Verilog)
Workshop:	VLSI System Design – MYTH Workshop
Date:	July 31, 2020 (Workshop) / June 2026 (Submission)

Day 3 Lab Coverage

- **Lab: Makerchip Platform**
- **Lab: Combinational Logic**
- **Lab: Vectors**
- **Lab: MUX**
- **Lab: Combinational Calculator**
- **Lab: Pipeline**
- **Lab: Counter**
- **Lab: Sequential Calculator**
- **Lab: Counter & Calculator in Pipeline**
- **Lab: 2-Cycle Calculator**
- **Lab: 2-Cycle Calculator with Validity**
- **Lab: Calculator with Single-Value Memory**

Contents

1 Workshop Overview

The RISC-V Based MYTH (Microprocessor for You in Thirty Hours) Workshop, Day 3, covers digital logic design using **TL-Verilog** inside the **Makerchip IDE**. The day progresses from foundational gate-level logic through combinational circuits, sequential circuits, pipelined logic, and culminates in a multi-cycle pipelined calculator with memory – forming the building blocks for the RISC-V CPU designed in Days 4 and 5.

1.1 Day 3 Agenda

1. Logic Gates
2. Makerchip Platform
3. Combinational Logic
4. Sequential Logic
5. Pipelined Logic
6. State

1.2 Theoretical Background

1.2.1 Logic Gates

The seven fundamental logic gates form the basis of all digital circuits. Their symbols, boolean representations, and Verilog equivalents are shown in Figure 1.

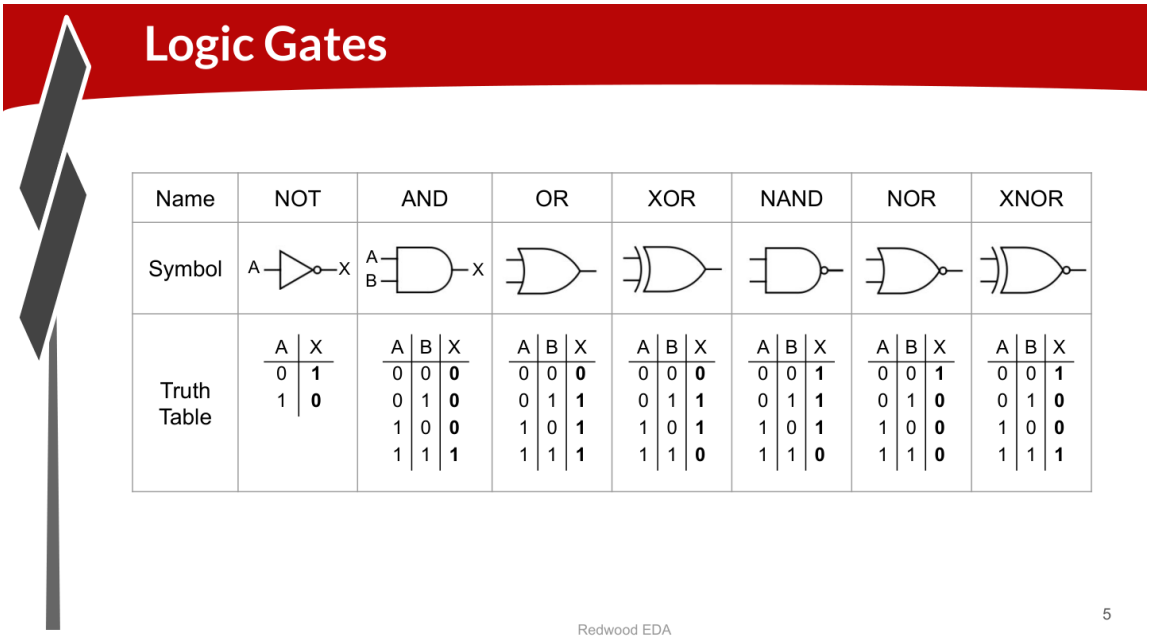


Figure 1: Logic Gates – Truth Tables and Gate Symbols

1.2.2 Combinational Circuits

A combinational circuit's output depends only on its current inputs, with no memory element. The full adder shown in Figure 2 computes sum S and carry-out C_{out} using

XOR, AND, and OR gates.

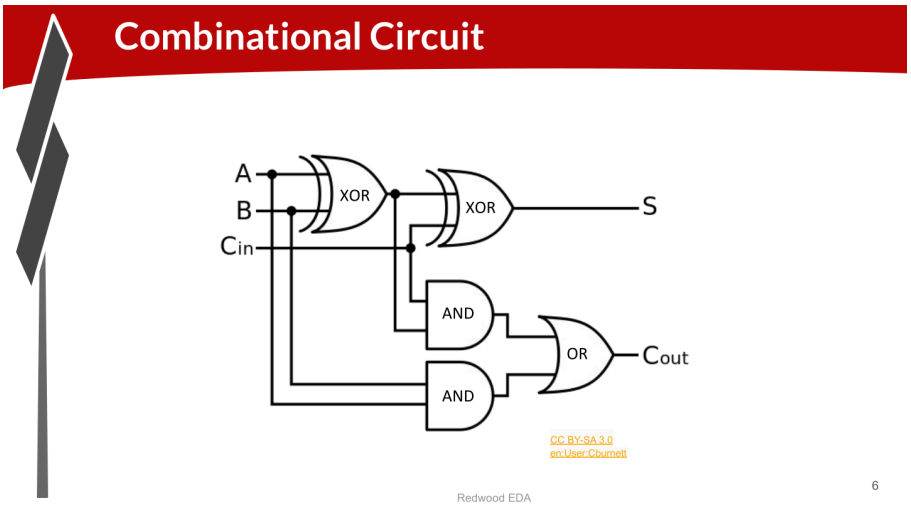


Figure 2: Full Adder – Combinational Circuit Example

1.2.3 Boolean Operators in Verilog / TL-Verilog

Operation	Bool. Arithmetic	Bool. Calculus	Verilog
NOT	$\overline{A}$	$\neg A$	`~A` or `!A`
AND	$A \cdot B$	$A \wedge B$	`A & B` or `&&`
OR	$A + B$	$A \vee B$	`A | B` or `||`
XOR	$A \oplus B$	$A \oplus B$	`A ^ B`
NAND	$\overline{A \cdot B}$	$\neg(A \wedge B)$	`!(A & B)`
NOR	$\overline{A + B}$	$\neg(A \vee B)$	`!(A | B)`
XNOR	$\overline{A \oplus B}$	$\neg(A \oplus B)$	`!(A ^ B)`

1.2.4 Multiplexer (MUX)

A multiplexer selects one of its inputs based on a select signal. In TL-Verilog the ternary operator is used:

```
1 assign f = s ? X1 : X2;
```

Listing 1: MUX in TL-Verilog using Ternary Operator

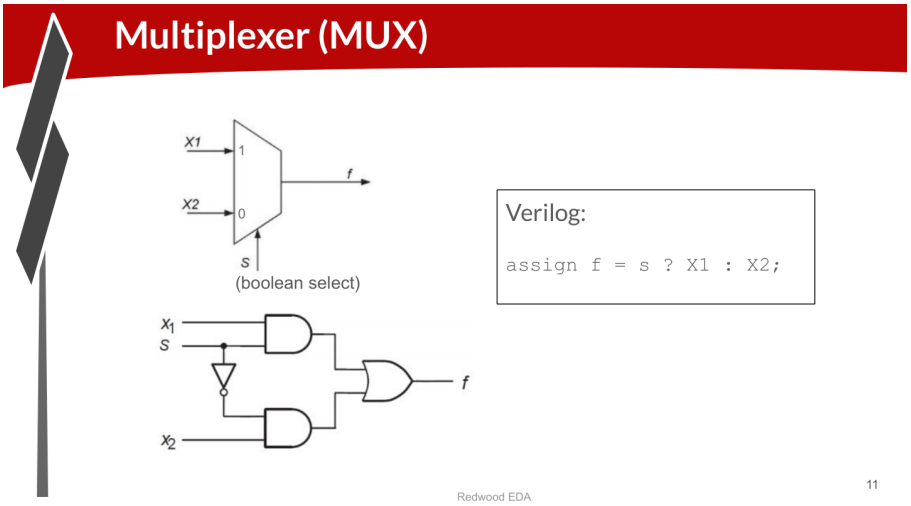


Figure 3: Multiplexer (MUX) – Symbol, Gate Implementation, and Verilog Code

1.2.5 Makerchip IDE

Makerchip is a free, browser-based IDE for TL-Verilog hardware design. It provides an integrated editor, waveform viewer, circuit diagram viewer, and simulation all in one environment.

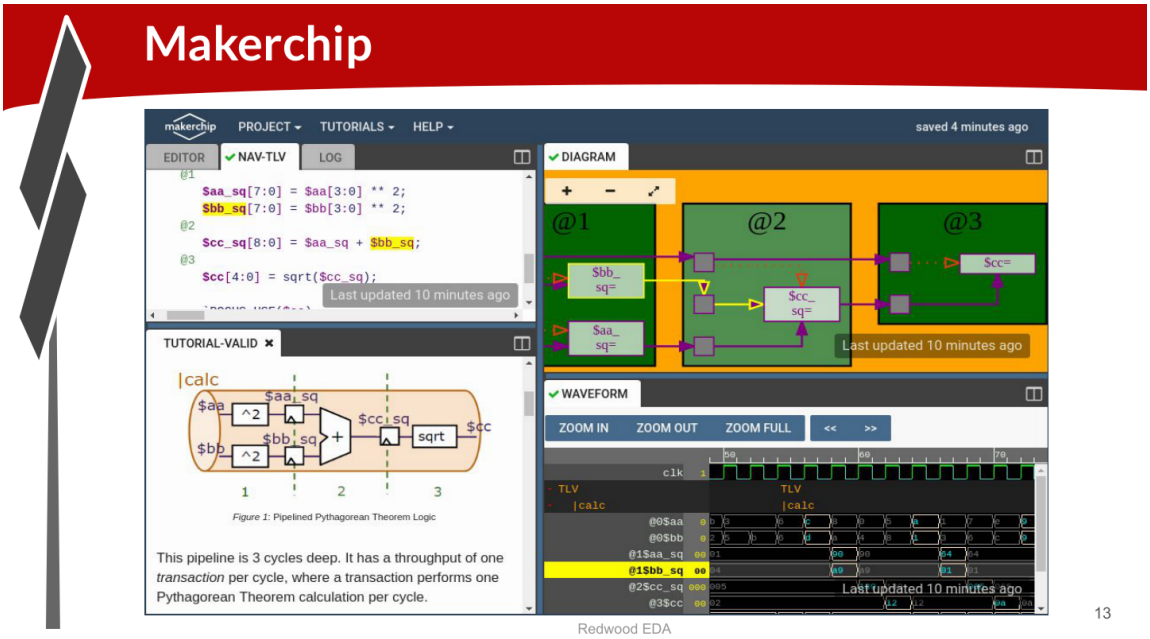


Figure 4: Makerchip IDE – Showing the Pipelined Pythagorean Theorem Example

1.2.6 Sequential Logic

Sequential circuits use D flip-flops, which capture "next state" on the rising clock edge. A reset signal brings the circuit to a known state on startup.

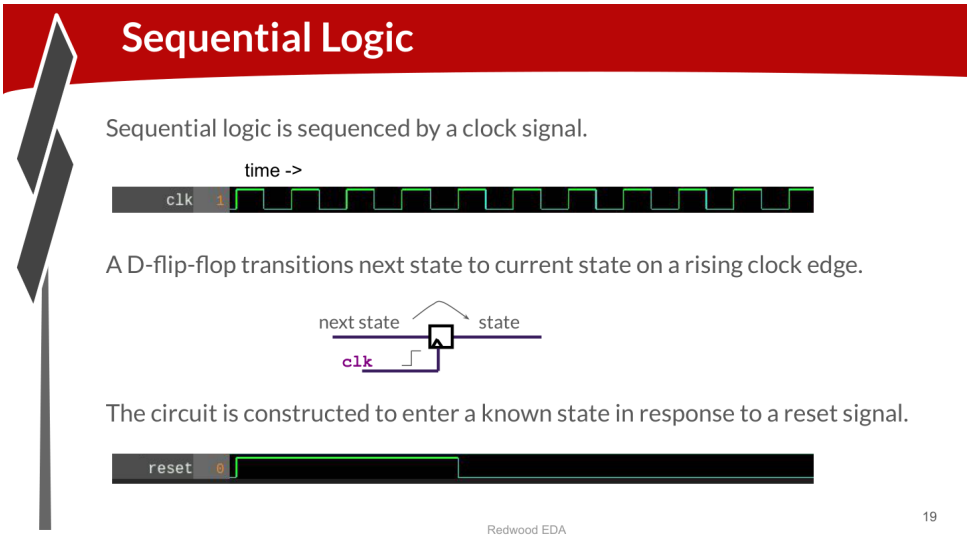


Figure 5: Sequential Logic – D Flip-Flop, Clock and Reset Signals

The Fibonacci series provides a simple sequential logic example where each value equals the sum of the two preceding values $(1, 1, 2, 3, 5, 8, 13, \ldots)$.

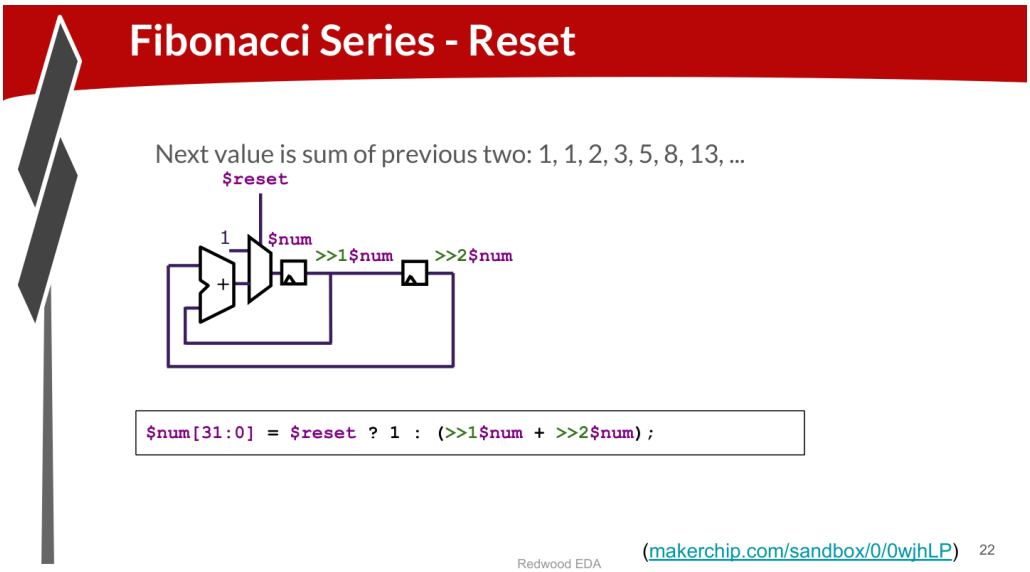


Figure 6: Fibonacci Series with Reset in TL-Verilog

```
1 $num[31:0] = $reset ? 1 : (>>1$num + >>2$num);
```

Listing 2: Fibonacci Series with Reset in TL-Verilog

Here, `>>1$num` references the value of `$num` from one cycle ago, and `>>2$num` references two cycles ago.

1.2.7 Pipelined Logic

TL-Verilog makes pipeline design much simpler than SystemVerilog. A computation is split into pipeline stages using `@N` markers. The language automatically handles pipeline registers – retiming a signal requires only changing its `@` number.

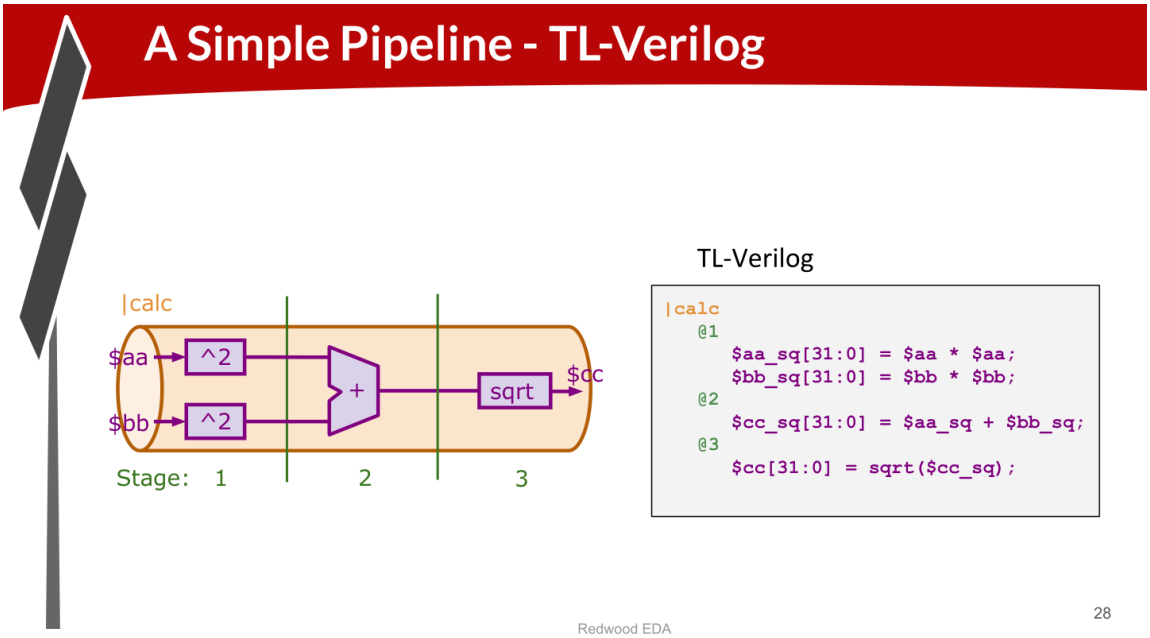


Figure 7: SystemVerilog vs TL-Verilog – 3.5x Code Reduction for Pythagorean Pipeline

2 Lab: Makerchip Platform

Lab 1 – Makerchip Platform

Goal: Explore the Makerchip IDE by loading the Pythagorean Theorem example and navigating its four panels: Editor, NAV-TLV, Diagram, and Waveform.

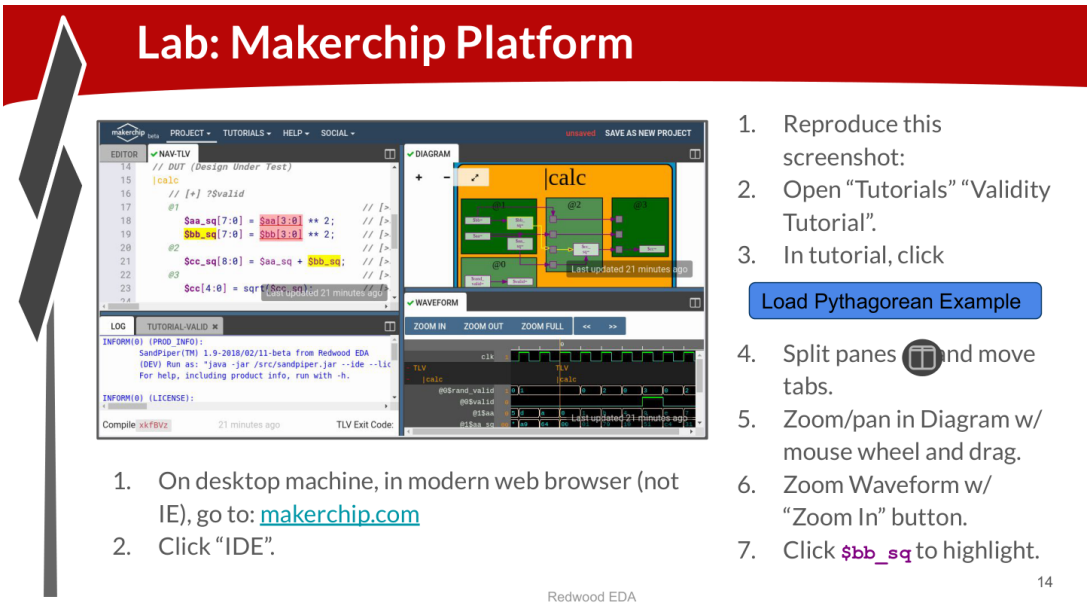
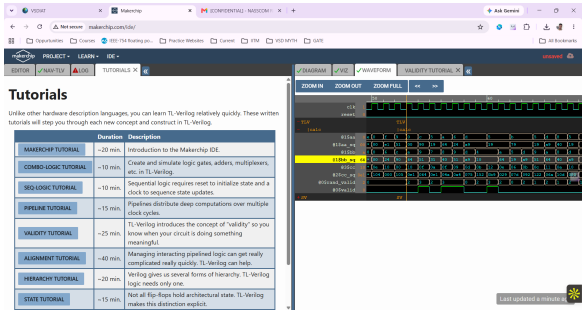


Figure 8: Lab Instructions – Makerchip Platform Navigation

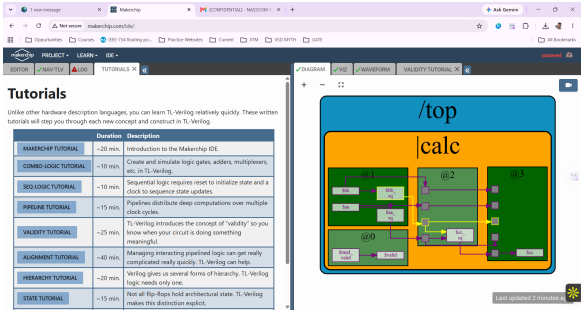
Steps Performed

1. Navigated to `https://makerchip.com` and clicked "IDE"
2. Opened **Tutorials** → **Validity Tutorial**
3. Clicked **Load Pythagorean Example** inside the tutorial
4. Explored split panes by moving the Editor, Diagram, and Waveform tabs
5. Zoomed and panned the Diagram using mouse wheel and drag
6. Zoomed the Waveform using the "Zoom In" button
7. Clicked on `$bb_sq` signal to highlight it in the diagram

Screenshots



(a) Makerchip IDE – Editor and Diagram panels



(b) Makerchip IDE – Waveform and signal highlighting

Figure 9: Lab: Makerchip Platform – Student Screenshots

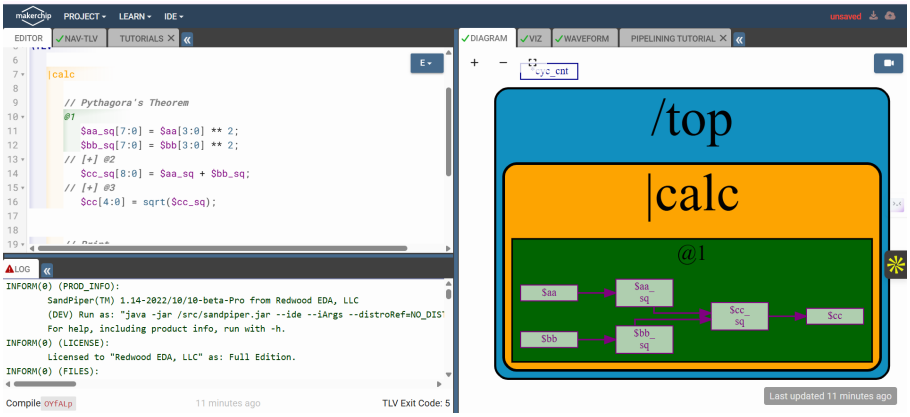


Figure 10: Makerchip IDE – Full View

Key Observations

The Makerchip IDE compiles TL-Verilog using the SandPiper(TM) compiler and provides real-time waveform simulation. Signals can be clicked in the NAV-TLV or Diagram panes to highlight their waveform traces. The Pythagorean Theorem example demonstrates a 3-stage pipeline computing $c = \sqrt{a^2 + b^2}$.

3 Lab: Combinational Logic

Lab 2 – Combinational Logic

Goal: Implement basic logic gates in TL-Verilog: (A) Inverter, (B) 2-input gates (AND, OR, XOR). Observe auto-generated diagrams and waveforms.

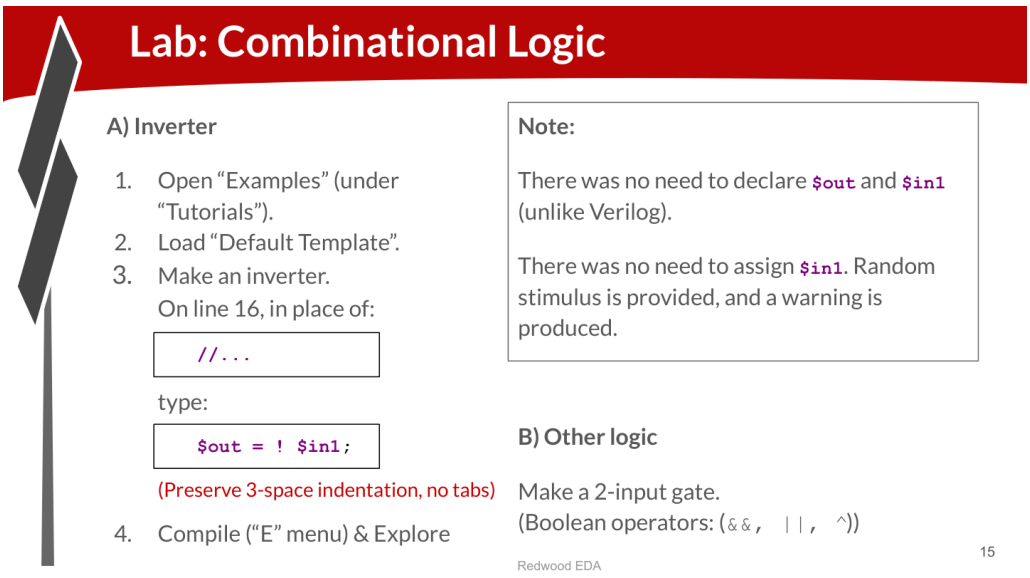


Figure 11: Lab Instructions – Combinational Logic

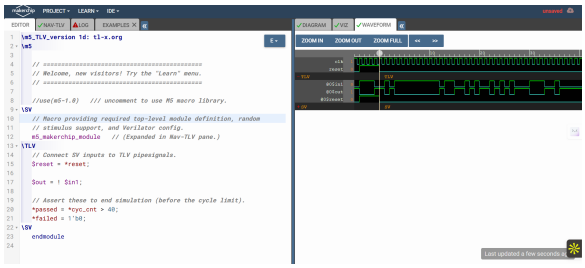
A) Inverter (NOT Gate)

```
1 \TLV
2    $out = ! $in1;
```

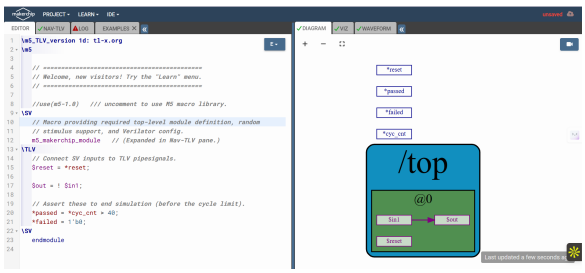
Listing 3: Inverter in TL-Verilog

Key observations:

- No need to declare `$out` or `$in1` (unlike Verilog)
- `$in1` receives random stimulus automatically; a warning is produced
- The diagram shows the NOT gate with input and output wires



(a) NOT gate – Diagram view



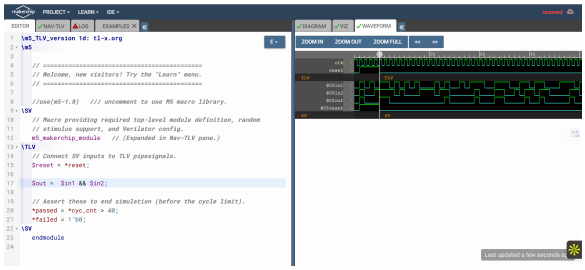
(b) NOT gate – Waveform view

Figure 12: Lab: Combinational Logic – Inverter

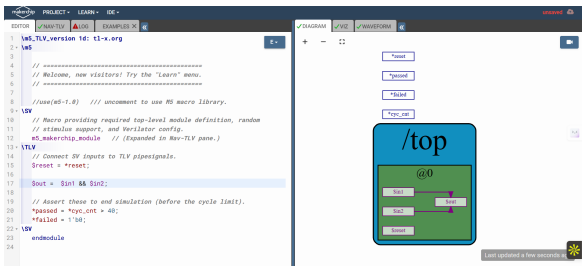
B) 2-Input Gates

```
// AND gate
$out = $in1 && $in2;

// OR gate
$out = $in1 || $in2;

// XOR gate
$out = $in1 ^ $in2;
```

Listing 4: AND / OR / XOR Gates in TL-Verilog

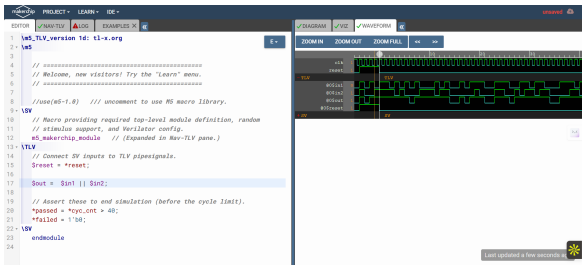


(a) AND gate – Diagram

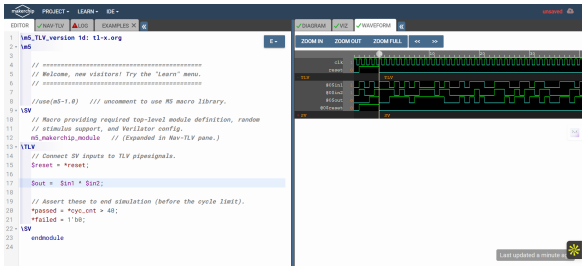


(b) AND gate – Waveform

Figure 13: Lab: Combinational Logic – AND Gate



(a) OR gate – Diagram and Waveform



(b) XOR gate – Diagram and Waveform

Figure 14: Lab: Combinational Logic – OR and XOR Gates

Key Observations

TL-Verilog eliminates boilerplate declarations. Signals are implicitly wire-type unless assigned with a flip-flop (`>>N`) operator. Random stimulus is automatically applied to undriven inputs, making rapid testing easy.

4 Lab: Vectors

Lab 3 – Vectors

Goal: Use bit-vector notation in TL-Verilog to perform arithmetic on multi-bit signals.

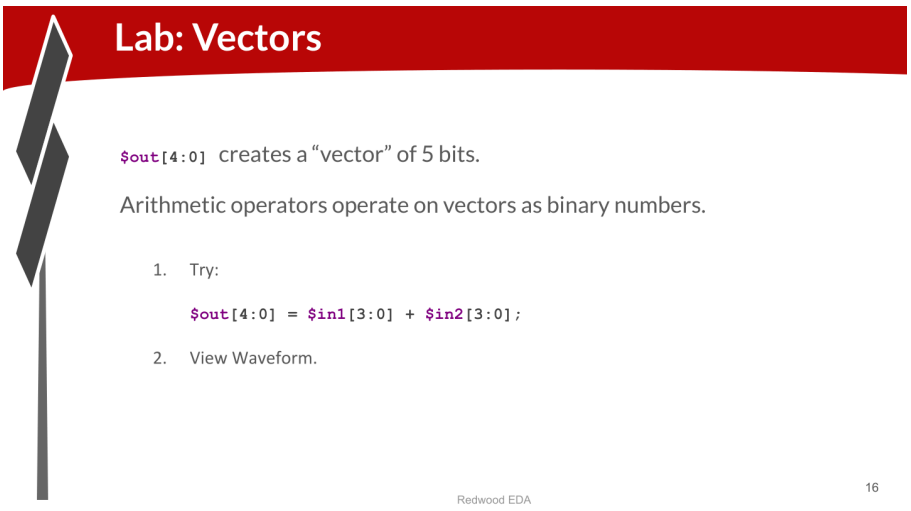


Figure 15: Lab Instructions – Vectors

Implementation

```
// 5-bit output = sum of two 4-bit inputs
$out[4:0] = $in1[3:0] + $in2[3:0];
```

Listing 5: Vector Addition in TL-Verilog

The bracket notation `[4:0]` creates a 5-bit vector. Arithmetic operators operate on vectors as binary numbers, automatically handling carry propagation.

Screenshot

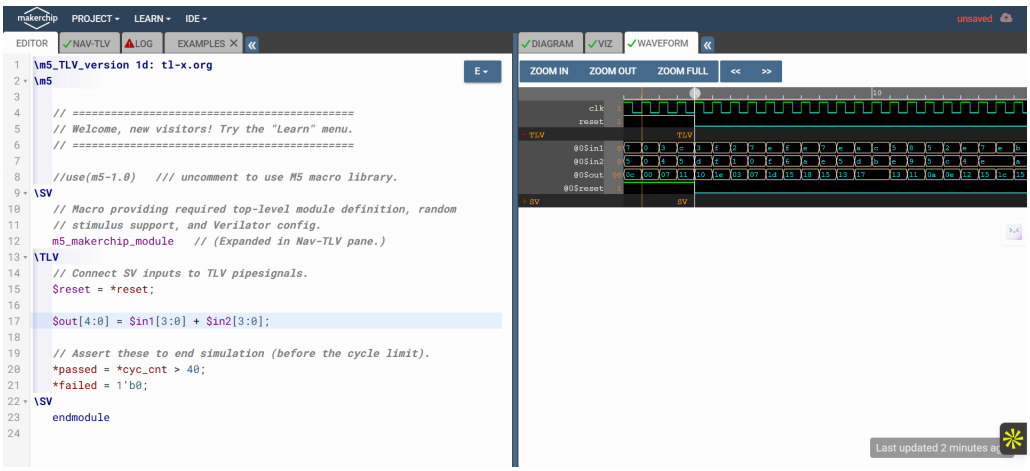


Figure 16: Lab: Vectors – Makerchip Simulation

Key Observations

Bit ranges can be implicit on the left-hand side when a signal is already assigned elsewhere. However, input signals with no prior assignment must have explicit bit ranges. The 5-bit output accommodates the carry from adding two 4-bit numbers ($\max 15 + 15 = 30 < 32 = 2^5$).

5 Lab: Multiplexer (MUX)

Lab 4 – Multiplexer

Goal: Implement a 2-to-1 multiplexer in TL-Verilog and extend it to operate on 8-bit vectors.

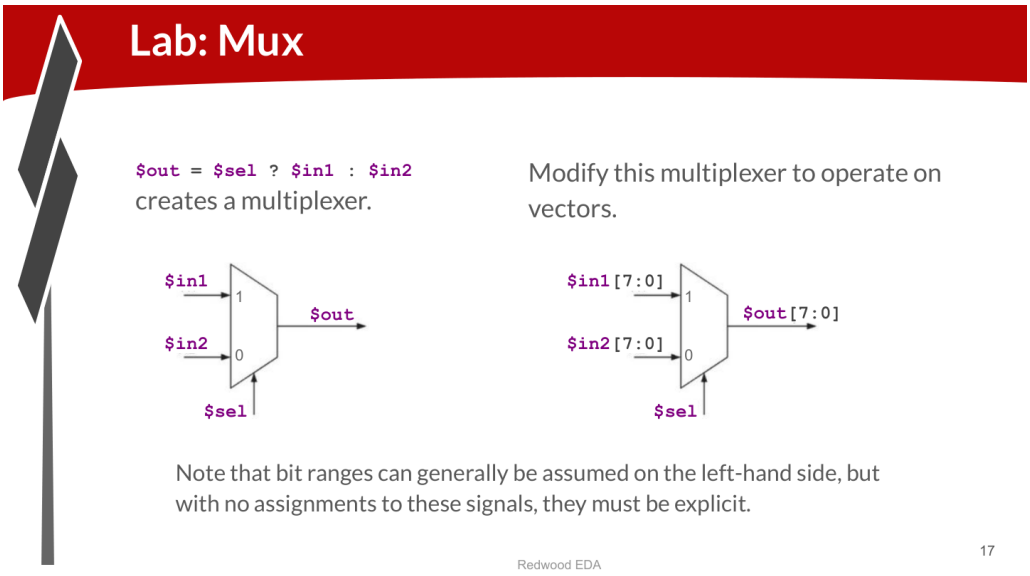


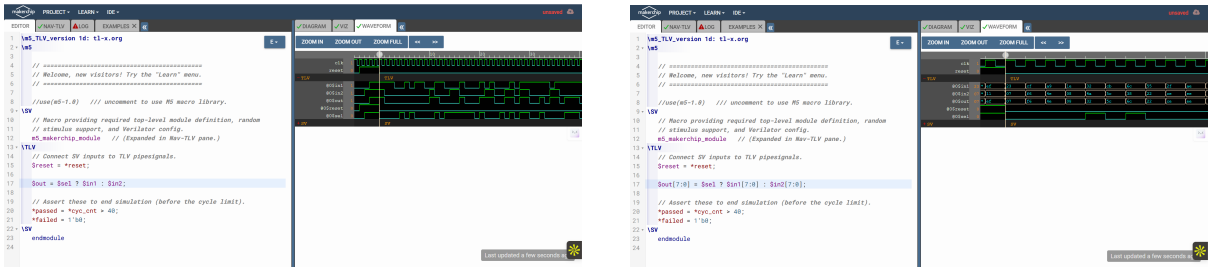
Figure 17: Lab Instructions – MUX (scalar and vector versions)

Implementation

```
// Scalar MUX
$out = $sel ? $in1 : $in2;

// 8-bit Vector MUX
$out[7:0] = $sel ? $in1[7:0] : $in2[7:0];
```

Listing 6: Scalar and Vector MUX in TL-Verilog

Screenshots



(a) Scalar MUX – Diagram (b) Vector MUX – Diagram and Waveform

Figure 18: Lab: MUX – Scalar and Vector Implementations

Key Observations

When `$sel = 1`, the output equals `$in1`; when `$sel = 0`, it equals `$in2`. For vector MUX, explicit bit ranges are required on input signals since they have no prior assignments. The ternary operator `?:` maps directly to a hardware MUX and is the idiomatic TL-Verilog way to express conditional selection.

6 Lab: Combinational Calculator

Lab 5 – Combinational Calculator

Goal: Implement a 4-function calculator (+, -, *, /) using a MUX to select the operation based on a 2-bit opcode `$op[1:0]`.

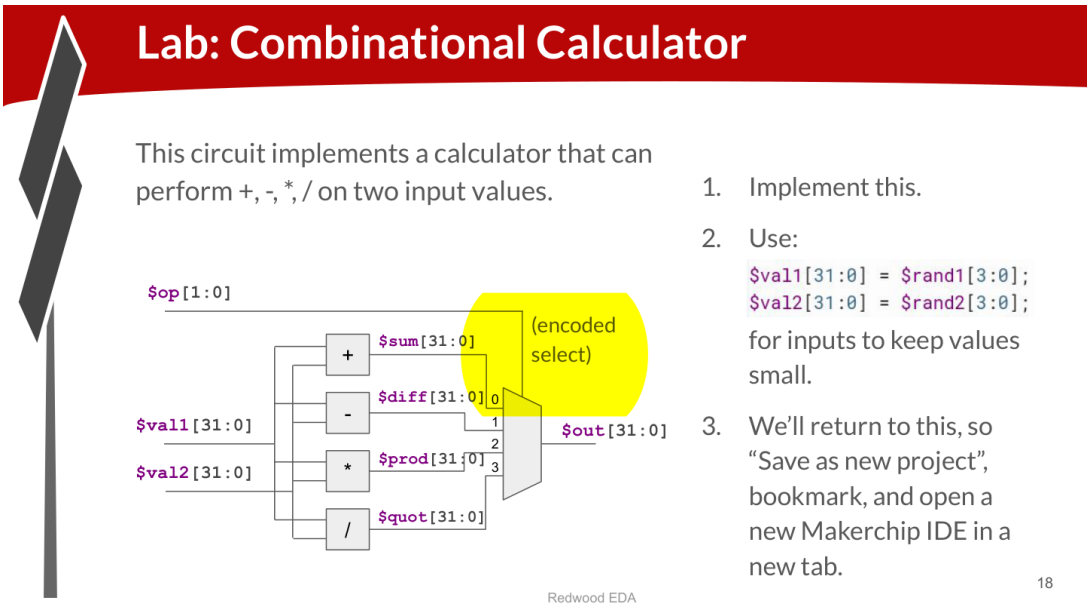


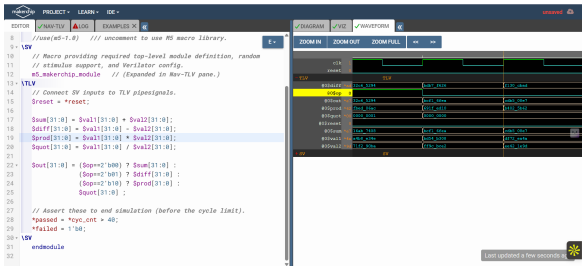
Figure 19: Lab Instructions – Combinational Calculator Architecture

Implementation

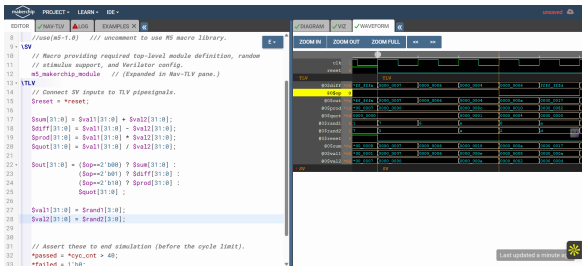
```
\TLV
   $val1[31:0] = $rand1[3:0];
   $val2[31:0] = $rand2[3:0];

   $sum[31:0]  = $val1 + $val2;
   $diff[31:0] = $val1 - $val2;
   $prod[31:0] = $val1 * $val2;
   $quot[31:0] = $val1 / $val2;

   $out[31:0] = $op[1:0] == 2'b00 ? $sum  :
                $op[1:0] == 2'b01 ? $diff :
                $op[1:0] == 2'b10 ? $prod :
                $quot;
```

Listing 7: Combinational Calculator in TL-Verilog

Screenshots



(a) Combinational Calculator – Editor



(b) Combinational Calculator – Diagram

Figure 20: Lab: Combinational Calculator – Code and Diagram

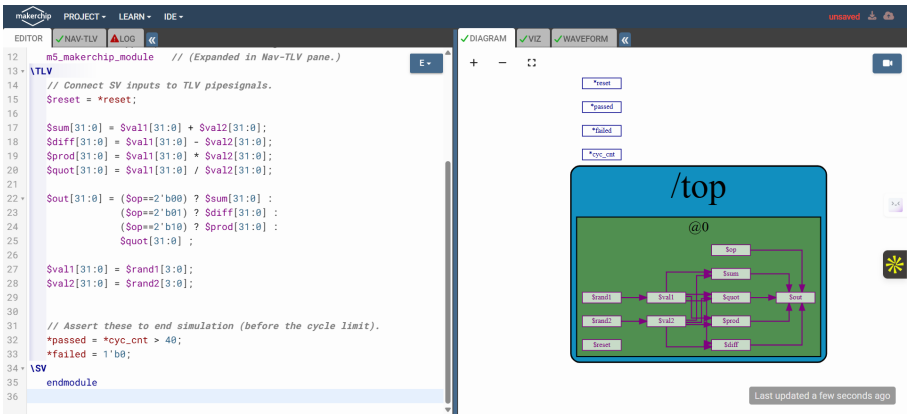


Figure 21: Combinational Calculator – Waveform showing all four operations

The saved Verilog code from this lab (`comb.v`) is shown below (the Pythagorean pipeline starter template used in the Makerchip session):

```
\m4_TLV_version 1d: tl-x.org
\SV
   `include "sqrt32.v";
   m4_makerchip_module
\TLV
   |calc
      // Pythagoras's Theorem
      @1
         $aa_sq[7:0] = $aa[3:0] ** 2;
         $bb_sq[7:0] = $bb[3:0] ** 2;
      @2
         $cc_sq[8:0] = $aa_sq + $bb_sq;
      @3
         $cc[4:0] = sqrt($cc_sq);

      // Print
         \SV_plus
            always_ff @(posedge clk) begin
               $display("sqrt((%2d^2) + (%2d^2)) = %2d",
                        $aa, $bb, $cc);
            end
   // Stop simulation.
```

```
   *passed = *cyc_cnt > 40;
\SV
endmodule
```

Listing 8: comb.v – Saved TL-Verilog Code from Lab

Key Observations

Random inputs `$rand1[3:0]` and `$rand2[3:0]` are used to keep operand values small (0–15), avoiding overflow. The chained ternary operator maps to a 4-to-1 encoded MUX. This project is saved as a Makerchip sandbox for reuse in subsequent labs.

7 Lab: Counter

Lab 6 – Free-Running Counter

Goal: Design a free-running 32-bit counter that resets to 1 on `$reset` and increments by 1 each cycle.

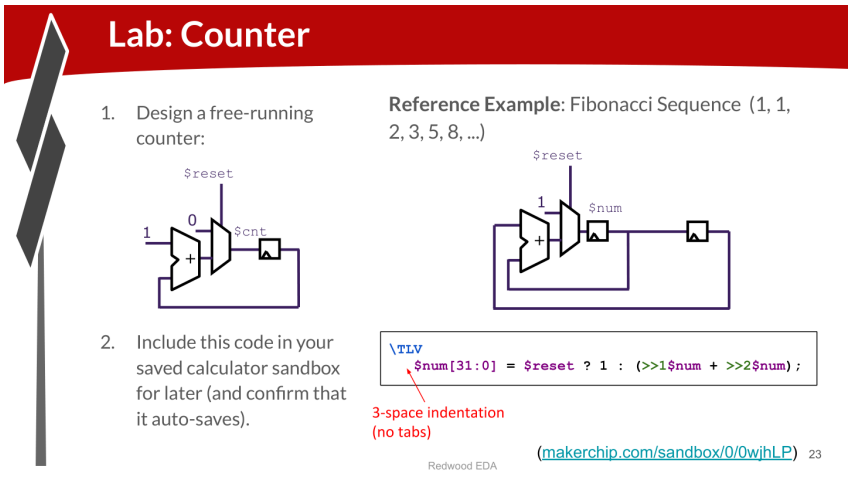


Figure 22: Lab Instructions – Counter (reference: Fibonacci sequence pattern)

Implementation

```
\TLV
   // Free-running counter
   $cnt[31:0] = $reset ? 0 : (>>1$cnt + 1);

   // Reference: Fibonacci Sequence (included in same sandbox)
   $num[31:0] = $reset ? 1 : (>>1$num + >>2$num);
```

Listing 9: Free-Running Counter in TL-Verilog

The counter uses the `>>1` operator to reference the previous cycle's value of `$cnt`. On reset, it initialises to 0; otherwise it increments.

Screenshot

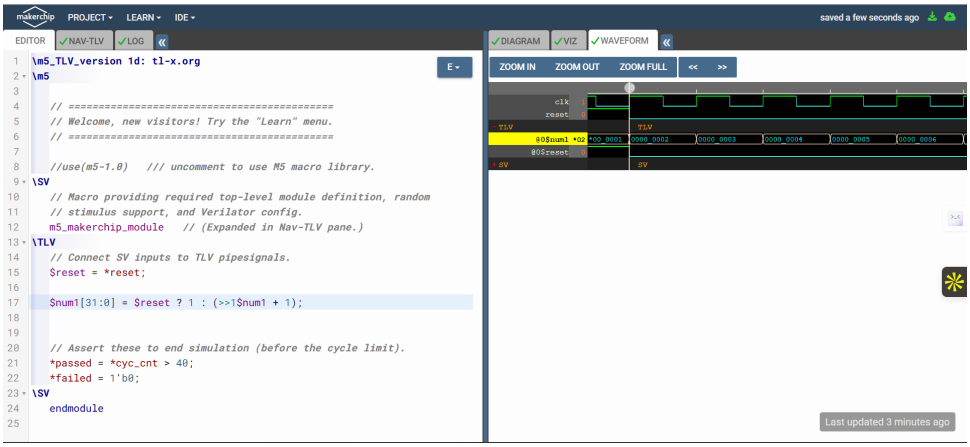


Figure 23: Lab: Counter – Makerchip Waveform showing incrementing counter

Key Observations

The `>>1$cnt` syntax in TL-Verilog denotes "`$cnt` from 1 cycle ago" – this is the pipeline staging operator that also serves as the sequential feedback operator. The counter code is included in the saved calculator sandbox so it can be used in later labs.

8 Lab: Sequential Calculator

Lab 7 – Sequential Calculator

Goal: Extend the combinational calculator so that the output of each calculation becomes `$val1` for the next calculation, mimicking how a real calculator "remembers" the previous result.

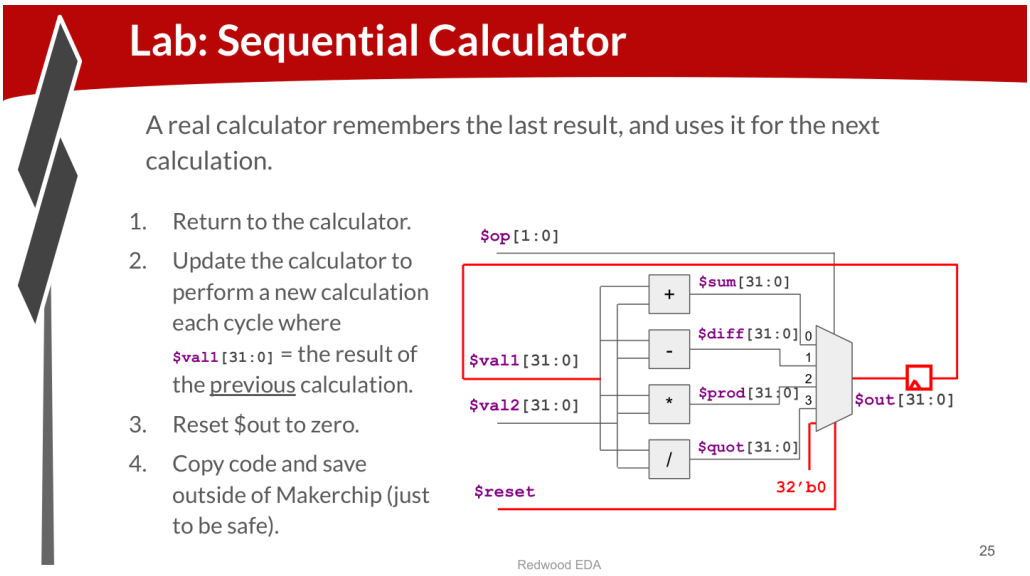


Figure 24: Lab Instructions – Sequential Calculator

Implementation

```
\TLV
   $val1[31:0] = $reset ? 32'b0 : >>1$out;
   $val2[31:0] = $rand2[3:0];

   $sum[31:0]  = $val1 + $val2;
   $diff[31:0] = $val1 - $val2;
   $prod[31:0] = $val1 * $val2;
   $quot[31:0] = $val1 / $val2;

   $out[31:0] = $reset ? 32'b0 :
                ($op[1:0] == 2'b00) ? $sum  :
                ($op[1:0] == 2'b01) ? $diff :
                ($op[1:0] == 2'b10) ? $prod :
                $quot;
```

Listing 10: Sequential Calculator in TL-Verilog

Key Observations

The key change from the combinational calculator is `$val1[31:0] = $reset ? 32'b0 : >>1$out;` – this creates a feedback loop where the previous output feeds the next calculation's first operand. On reset, the output is forced to zero with the literal `32'b0`.

9 Lab: Counter and Calculator in Pipeline

Lab 8 – Counter and Calculator in Pipeline

Goal: Wrap the counter and calculator logic inside a `|calc` pipeline at stage `@1`. Verify correct operation in the diagram and waveform.

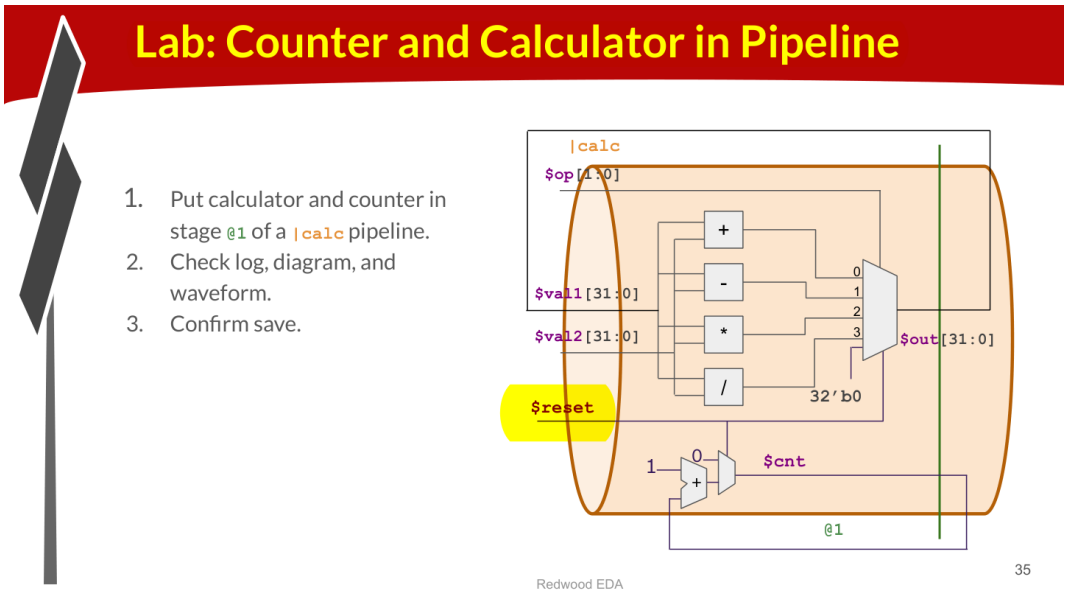


Figure 25: Lab Instructions – Counter and Calculator in a Pipeline Stage

Implementation

```
\TLV
   |calc
      @1
         // Free-running counter
         $cnt[31:0] = $reset ? 0 : (>>1$cnt + 1);

         // Sequential Calculator
         $val1[31:0] = $reset ? 32'b0 : >>1$out;
         $val2[31:0] = $rand2[3:0];

         $sum[31:0]  = $val1 + $val2;
         $diff[31:0] = $val1 - $val2;
         $prod[31:0] = $val1 * $val2;
         $quot[31:0] = $val1 / $val2;

         $out[31:0] = $reset ? 32'b0 :
                      ($op[1:0] == 2'b00) ? $sum  :
                      ($op[1:0] == 2'b01) ? $diff :
                      ($op[1:0] == 2'b10) ? $prod :
                      $quot;
```

Listing 11: Counter and Calculator Placed Inside Pipeline Stage @1

Key Observations

Placing logic inside a named pipeline (`|calc`) and a stage (`@1`) prepares it for later pipelining and retiming. The `|calc` pipeline becomes visible in the NAV-TLV navigator and the diagram view. No functional change occurs yet – all logic is in stage 1, so no flip-flops are inserted between stages.

10 Lab: 2-Cycle Calculator

Lab 9 – 2-Cycle Calculator

Goal: Retime the MUX to pipeline stage `@2`, producing a calculator that computes every other cycle. Use a 1-bit counter as a validity signal, and modify the feedback to use `>>2` (two cycles ago).

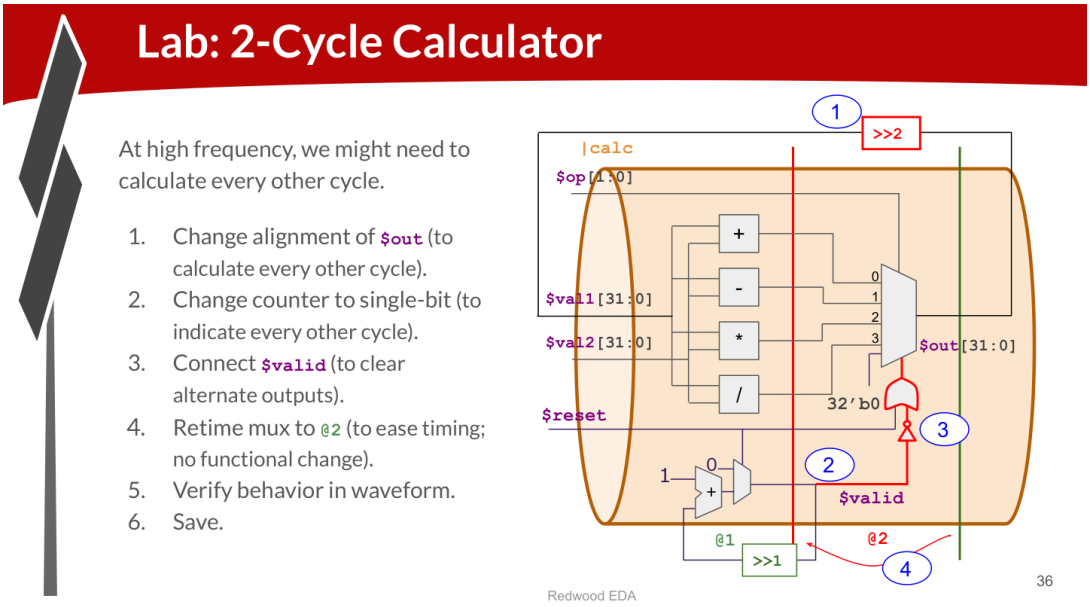


Figure 26: Lab Instructions – 2-Cycle Calculator

Implementation

```
\TLV
   |calc
      @1
         $valid[0:0] = $reset ? 1'b0 : (>>1$valid + 1);

         $val1[31:0] = $reset ? 32'b0 : >>2$out;
         $val2[31:0] = $rand2[3:0];

         $sum[31:0]  = $val1 + $val2;
         $diff[31:0] = $val1 - $val2;
         $prod[31:0] = $val1 * $val2;
         $quot[31:0] = $val1 / $val2;

      @2
         $out[31:0] = ($reset || !$valid) ? 32'b0 :
                      ($op[1:0] == 2'b00) ? $sum  :
                      ($op[1:0] == 2'b01) ? $diff :
                      ($op[1:0] == 2'b10) ? $prod :
                      $quot;
```

Listing 12: 2-Cycle Calculator in TL-Verilog

Key Changes

- **1-bit valid counter:** `$valid[0:0]` toggles every cycle, indicating every-other-cycle computation
- **MUX retimed to @2:** No functional change, but the flip-flop between @1 and @2 allows a higher operating frequency
- **Feedback uses ¿¿2:** Because the result is in @2, the feedback must reach back two cycles to set `$val1` in the correct @1 cycle
- **Output cleared on invalid cycles:** `!$valid` zeros the output on odd cycles when no new computation occurs

Key Observations

Retiming is purely a timing/frequency optimisation. The behaviour, as seen in the waveform, is identical – every other valid output is the correct result of applying the selected operation to `$val1` and `$val2`.

11 Lab: 2-Cycle Calculator with Validity

Lab 10 – 2-Cycle Calculator with Validity

Goal: Replace the zero-clearing approach with TL-Verilog's `?` validity construct. Use `$valid_or_reset` as the when-condition for the calculation block instead of zeroing `$out`.

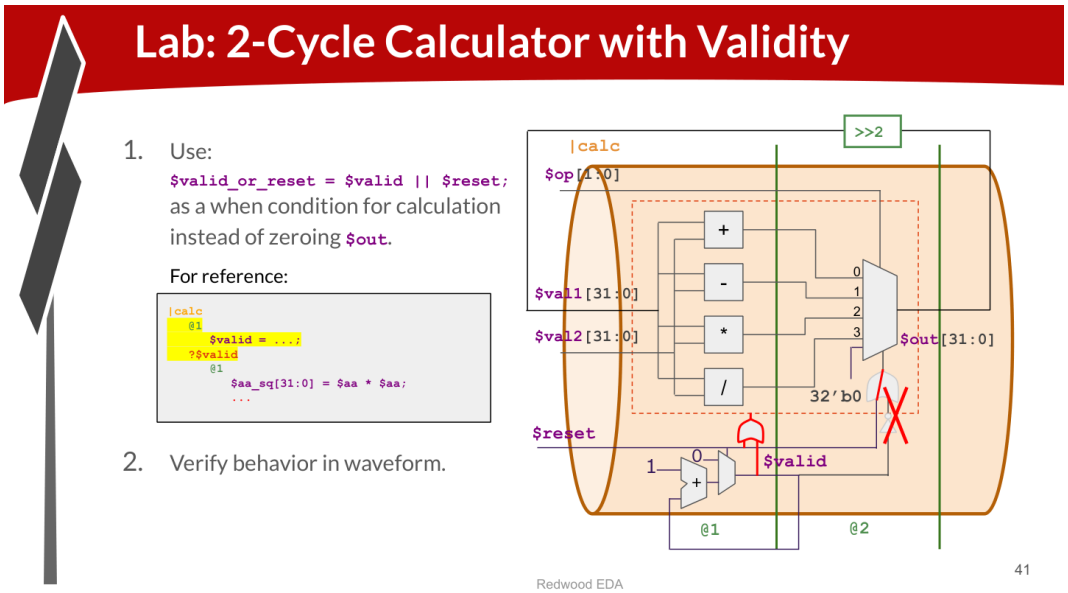


Figure 27: Lab Instructions – 2-Cycle Calculator with Validity

Implementation

```
\TLV
   |calc
      @1
         $valid = $reset ? 1'b0 : (>>1$valid + 1);
         $valid_or_reset = $valid || *reset;

      ?$valid_or_reset
         @1
            $val1[31:0] = $reset ? 32'b0 : >>2$out;
            $val2[31:0] = $rand2[3:0];

            $sum[31:0]  = $val1 + $val2;
            $diff[31:0] = $val1 - $val2;
            $prod[31:0] = $val1 * $val2;
            $quot[31:0] = $val1 / $val2;

         @2
            $out[31:0] = ($op[1:0] == 2'b00) ? $sum  :
                         ($op[1:0] == 2'b01) ? $diff :
                         ($op[1:0] == 2'b10) ? $prod :
                         $quot;
```

Listing 13: 2-Cycle Calculator with Validity in TL-Verilog

Benefits of Validity

- **Easier debug:** Signals marked invalid are shown in the waveform in a distinct way
- **Cleaner design:** No need to manually zero outputs on invalid cycles
- **Better error checking:** TL-Verilog verifies that signals under a `?` context are not used outside it
- **Automated clock gating:** SandPiper can automatically gate the clocks of flip-flops under an invalid when-condition, reducing power consumption

Key Observations

The `?` when-condition is a TL-Verilog construct with no SystemVerilog equivalent. It enables timing-abstract gating and is the recommended way to handle conditional computation in pipelines. The `$valid_or_reset` ensures the pipeline state is initialised correctly on reset even when `$valid` is low.

12 Lab: Calculator with Single-Value Memory

Lab 11 – Calculator with Single-Value Memory

Goal: Extend `$op` to 3 bits and add "mem" (store) and "recall" (load) operations to the calculator, using a single 32-bit register as memory.

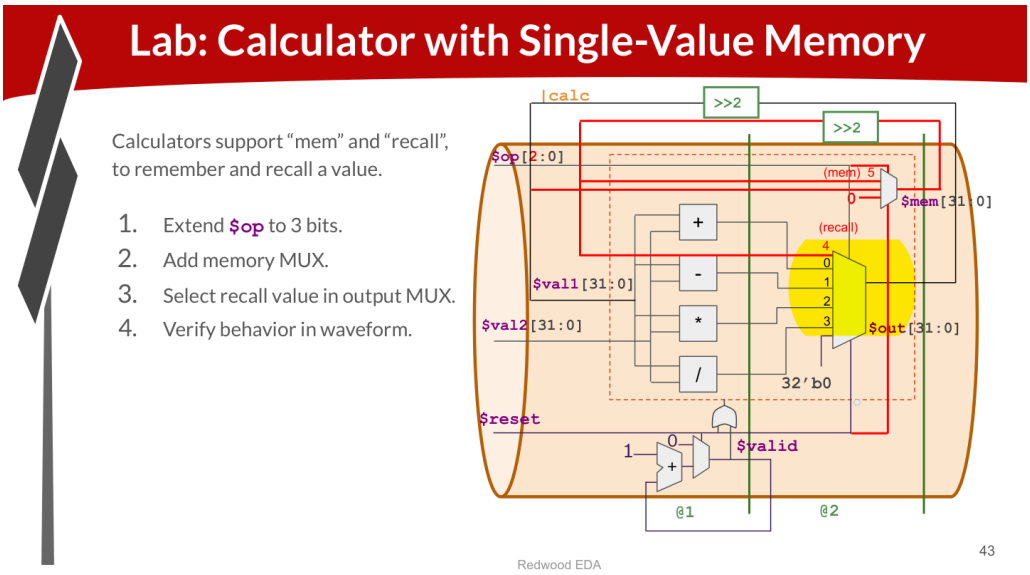


Figure 28: Lab Instructions – Calculator with Single-Value Memory

Implementation

```
\TLV
   |calc
      @1
         $valid = $reset ? 1'b0 : (>>1$valid + 1);
         $valid_or_reset = $valid || *reset;

      ?$valid_or_reset
         @1
            $val1[31:0] = $reset ? 32'b0 : >>2$out;
            $val2[31:0] = $rand2[3:0];

            $sum[31:0]  = $val1 + $val2;
            $diff[31:0] = $val1 - $val2;
            $prod[31:0] = $val1 * $val2;
            $quot[31:0] = $val1 / $val2;

         @2
            // Single-value memory (mem/recall)
            $mem[31:0] = $reset ? 32'b0 :
                         ($op[2:0] == 3'b101) ? >>2$out : >>2$mem;

            $out[31:0] = ($op[2:0] == 3'b000) ? $sum  :
                         ($op[2:0] == 3'b001) ? $diff :
                         ($op[2:0] == 3'b010) ? $prod :
                         ($op[2:0] == 3'b011) ? $quot :
                         ($op[2:0] == 3'b100) ? >>2$mem :
```

```
27                                    32'b0;
```

Listing 14: Calculator with Single-Value Memory in TL-Verilog

Operation Encoding

$op[2:0]	Operation	Description
3'b000	ADD	`$out = $val1 + $val2`
3'b001	SUB	`$out = $val1 - $val2`
3'b010	MUL	`$out = $val1 * $val2`
3'b011	DIV	`$out = $val1 / $val2`
3'b100	RECALL	`$out = $mem`
3'b101	MEM	`$mem = $out` (stores current output)

Key Observations

The single-value memory `$mem` is itself a sequential register – it retains its value until explicitly written by the MEM operation. The `>>2$mem` feedback is required because `$mem` is in stage @2 and we are already in @2, so its previous value is 2 cycles back.

13 Summary of Labs Completed

#	Lab Name	Status	Key Learning
1	Makerchip Platform	Complete	IDE navigation, diagram, waveform
2	Combinational Logic	Complete	NOT, AND, OR, XOR gates in TL-V
3	Vectors	Complete	Multi-bit arithmetic with `[N:M]`
4	Multiplexer (MUX)	Complete	Ternary operator for selection
5	Combinational Calculator	Complete	4-op calculator with encoded MUX
6	Counter	Complete	`>>1` feedback for sequential logic
7	Sequential Calculator	Complete	`>>1$out` feedback for state
8	Counter & Calc in Pipeline	Complete	`|pipe @N` staging construct
9	2-Cycle Calculator	Complete	Retiming, `>>2`, validity signal
10	2-Cycle Calc + Validity	Complete	`?$valid` construct, clock gating
11	Single-Value Memory	Complete	Memory MUX, 3-bit opcode

14 Key TL-Verilog Concepts Covered

TL-Verilog Quick Reference – Day 3

- **Signal prefix:** `$` for pipeline signals, `*` for system signals (`*reset`, `*clk`)
- **Pipeline staging:** `@N` assigns a signal to pipeline stage N
- **Pipeline reference:** `>>N$sig` = value of `$sig` from N cycles/stages ago
- **Combinational MUX:** `$out = $sel ? $in1 : $in2;`
- **Chained ternary (4-way MUX):** use nested `?:` operators
- **Vector declaration:** `$sig[N:M]` for bits N down to M
- **Validity context:** `?$valid` gates a block of logic
- **Pipeline name:** `\|pipename` groups related stages
- **No declarations needed:** signals auto-declared from assignments
- **Retiming:** change `@N` number; SandPiper handles all register additions

15 Conclusion

Day 3 of the RISC-V MYTH Workshop successfully covered the foundations of digital logic design in TL-Verilog using the Makerchip IDE. Starting from single-gate combinational logic, the labs progressively built up to a pipelined 2-cycle calculator with memory

operations – demonstrating the power of TL-Verilog's abstraction.

The **key insight** is that TL-Verilog's pipeline staging (`@N`) and staging reference (`>>N`) operators allow engineers to describe hardware at a higher level of abstraction, with the compiler automatically handling all the boilerplate register declarations and connections that SystemVerilog requires manually. This ˜3.5× code reduction makes the design far less error-prone, especially for retiming.

These concepts – combinational logic, sequential state, pipelining, and validity – form the direct building blocks for the RISC-V CPU core that will be constructed in Days 4 and 5.